UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	Muhammad Ilham Fajri (2428240082)
Kelas	SI5B
Judul project / topik API	**Tugas 1 RESTful API - Asuransi**
Tautan repositori	https://github.com/IlhamFajrii/tugas1-restful-2428240082
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	22 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

> **Sebelum mengambil screenshot Runner**
> Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:
RESTful API Asuransi digunakan untuk mengelola data polis melalui resource /insurance-policies. Operasi utama mencakup melihat semua polis, melihat detail polis berdasarkan ID, menambahkan polis, memperbarui polis, dan menghapus polis. Teknologi yang digunakan adalah Node.js dan Express.js, sedangkan Postman digunakan untuk menguji request dan respons JSON. Jalankan server pada port yang sesuai dengan konfigurasi project.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/insurance-policies	Tidak*	200	Menampilkan semua polis
2	GET	/insurance-policies/:id	Tidak*	200; 404 jika tidak ditemukan	Menampilkan satu polis berdasarkan ID
3	POST	/insurance-policies	Ya	201; 400/401 untuk input gagal	Menambahkan polis baru
4	PUT	/insurance-policies/:id	Ya	200; 404 jika ID tidak ditemukan	Memperbarui data polis
5	DELETE	/insurance-policies/:id	Ya	200/204; 404 jika ID tidak ditemukan	Menghapus polis
6	GET	/insurance-policies?jenis=kesehatan	Tidak	200 jika filter diimplementasikan	Memfilter polis berdasarkan jenis
7					
8					

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat server API lokal

No	Nama variabel	Nilai (contoh)	Fungsi
2	api_key	Nilai berada di file .env	Header API key untuk request yang memerlukannya; jangan gunakan nilai rahasia di laporan
3	createdPolicyId	diisi otomatis oleh POST	ID polis baru untuk request Satu Polis, Ubah Polis, dan Hapus Polis

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:
Request dibagi menjadi dua folder seperti struktur Collection pada Postman. Folder A.Crud berisi lima request utama untuk melihat semua polis, menambahkan polis, melihat satu polis, mengubah polis, dan menghapus polis. Folder B.Skenario Gagal berisi empat request untuk memastikan API menangani ID yang sudah dihapus, permintaan tanpa API key, data tidak lengkap, dan JSON rusak. Jalankan request CRUD secara berurutan agar ID dari request Tambah Polis tersedia untuk request berikutnya.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	Semua Polis	GET	{{base_url}}/insurance-policies	A.Crud
2	Tambah Polis	POST	{{base_url}}/insurance-policies	A.Crud
3	Satu Polis	GET	{{base_url}}/insurance-policies/{{createdPolicyId}}	A.Crud
4	Ubah Polis	PUT	{{base_url}}/insurance-policies/{{createdPolicyId}}	A.Crud
5	Hapus Polis	DELETE	{{base_url}}/insurance-policies/{{createdPolicyId}}	A.Crud

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
6	GET ID Sudah Dihapus (404)	GET	{{base_url}}/insurance-policies/{{deletedPolicyId}}	B.Skenario Gagal
7	POST Tanpa API Key (401)	POST	{{base_url}}/insurance-policies	B.Skenario Gagal
8	POST Data Tidak Lengkap (400)	POST	{{base_url}}/insurance-policies	B.Skenario Gagal
9	POST JSON Rusak (400)	POST	{{base_url}}/insurance-policies	B.Skenario Gagal

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	Semua Polis	200 OK	Body berupa array JSON / daftar polis
2	Tambah Polis	201 Created	status=success dan data.id tersedia; simpan ID ke createdPolicyId
3	Satu Polis	200 OK	Objek data memuat ID polis yang diminta
4	Ubah Polis	200 OK	status=success dan ID pada respons sesuai createdPolicyId
5	Hapus Polis	200 OK atau 204 sesuai implementasi	Respons menunjukkan penghapusan berhasil
6	GET ID Sudah Dihapus (404)	404 Not Found	Respons JSON menunjukkan status/error
7	POST Tanpa API Key (401)	401 Unauthorized	Respons menyatakan API key tidak ada/tidak valid
8	POST Data Tidak Lengkap (400)	400 Bad Request	Respons menjelaskan data wajib belum lengkap
9	POST JSON Rusak (400)	400 Bad Request	Respons error JSON; request dikirim dengan raw body JSON tidak valid

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
 // Test script request GET: GET - Semua Polis
pm.test("Status code 200", function () {
    pm.response.to.have.status(200);
});
pm.test("Respons berupa array JSON", function () {
    const body = pm.response.json();
    pm.expect(body).to.be.an("array");
});
```

Penjelasan:
Script GET Semua Polis memeriksa dua hal. Assertion pertama memastikan server mengembalikan HTTP 200. Assertion kedua memastikan body respons dapat dibaca sebagai JSON dan berbentuk array, karena endpoint ini bertujuan menampilkan daftar polis. Dengan demikian, test memeriksa status HTTP sekaligus bentuk respons.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
 // Test script request POST: POST - Tambah Polis
pm.test("Status code 201", function () {
    pm.response.to.have.status(201);
});
pm.test("Respons sukses dan ID tersedia", function () {
    const body = pm.response.json();
    pm.expect(body.status).to.eql("success");
    pm.expect(body.data).to.have.property("id");
    pm.environment.set("createdPolicyId", body.data.id);
});
```

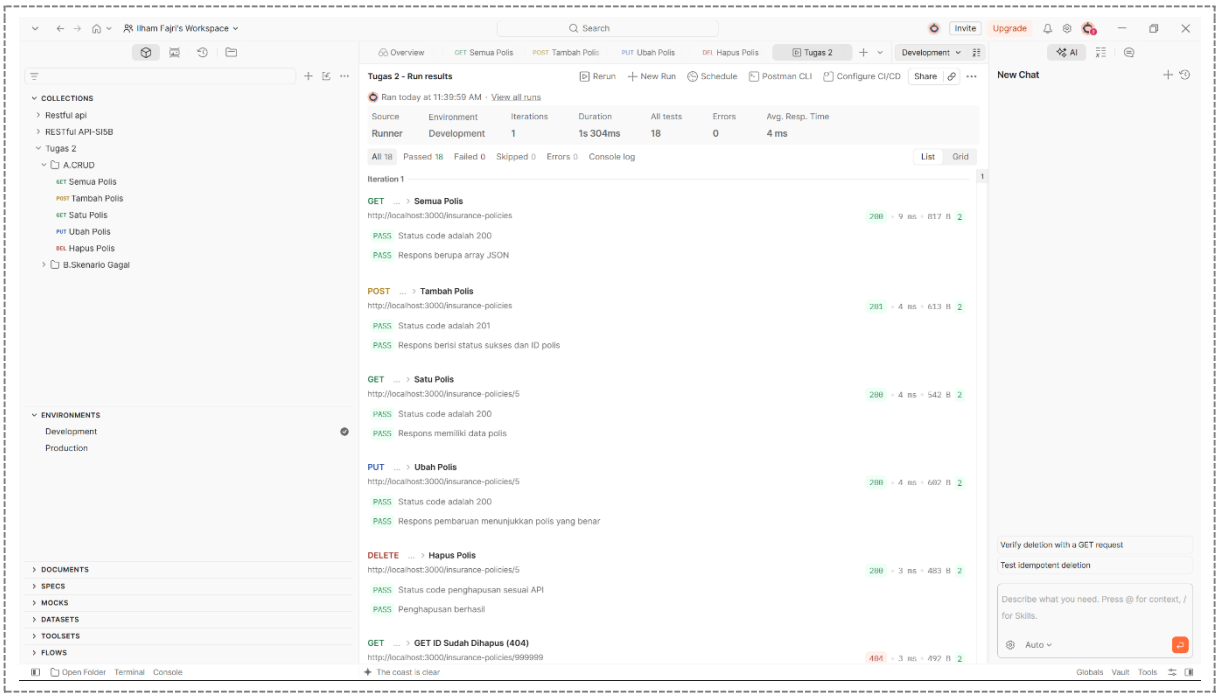
Penjelasan:
Script Tambah Polis memeriksa status 201, memastikan respons menyatakan operasi berhasil, dan memastikan data memiliki properti id. ID yang diterima disimpan sebagai createdPolicyId untuk dipakai request Satu Polis, Ubah Polis, dan Hapus Polis. Jika request lanjutan dijalankan sebelum Tambah Polis, variabel ID belum terisi sehingga test berikutnya dapat gagal.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:
Collection Runner digunakan untuk menjalankan seluruh sembilan request dan assertion dalam satu rangkaian. Sebelum menjalankan Runner, nyalakan server, pilih Environment yang benar, pastikan API key diatur untuk request yang memerlukannya, dan periksa urutan request. Target hasil akhir adalah seluruh assertion lulus dan jumlah test gagal 0; masukkan angka aktual sesuai hasil Runner setelah pengujian dijalankan.

4.2 Rekap angka hasil

Nama Collection	Tugas 1 RESTful API - Asuransi
Environment yang dipakai	Tugas 1 RESTful API - Asuransi
Jumlah request dijalankan	10 request (sesuaikan dengan Collection yang benar-benar dijalankan)
Jumlah test lulus	Target: seluruh test lulus
Jumlah test gagal	Target: 0
Tanggal dan jam pengujian	Isi sesuai waktu Collection Runner dijalankan

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	Jika test gagal sebelum perbaikan, catat pesan aktual dari Postman	Periksa status code, API key, validasi field wajib, dan format JSON	Sesuaikan request/script dengan respons API, lalu jalankan ulang sampai 0 failed	
2	—	—	—	—

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

Tidak Dikerjakan

Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:
tidak dikerjakan

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/IlhamFajrii/tugas1-restful-2428240082
Lokasi berkas Collection (.json)	Postman/Asuransi-API-Collection.json (buat/ekspor file ini dari Postman)
Lokasi berkas Environment (.json)	Postman/Asuransi-Local-Environment.json (ekspor tanpa nilai rahasia sungguhan)
Berkas Environment tidak memuat kunci rahasia sungguhan	Ya, setelah diperiksa sebelum dikumpulkan
Repositori dapat dibuka dosen	Ya, jika akses repositori publik tetap aktif

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Assertion status code paling berguna untuk memastikan setiap operasi mengembalikan kode HTTP yang sesuai, misalnya 201 saat menambah polis dan 404 saat ID tidak ditemukan. Assertion struktur JSON juga penting karena respons yang berstatus sukses tetap harus memiliki properti yang dibutuhkan.

Penjelasan:
Saya memilih assertion status code dan struktur JSON karena keduanya memeriksa dua hal penting sekaligus: apakah endpoint mengembalikan kode HTTP yang tepat dan apakah bentuk data respons sesuai kebutuhan. Contohnya, request POST harus mengembalikan status 201 dan ID polis yang dapat dipakai request selanjutnya. Pada skenario ID tidak ditemukan, assertion mengharapkan status 404 sehingga respons kesalahan juga diuji secara otomatis.

Pertanyaan 2

Pengujian manual perlu mengirim request satu per satu dan memeriksa hasilnya sendiri, sedangkan Collection Runner menjalankan rangkaian request dan assertion secara berurutan. Bagian yang perlu diperhatikan adalah menyimpan ID dari respons POST ke environment agar request berikutnya menggunakan ID yang benar; saya mengatasinya dengan variabel createdPolicyId.

Penjelasan:
Pengujian manual dilakukan dengan mengirim request satu per satu dan memeriksa respons secara langsung, sedangkan Collection Runner menjalankan banyak request secara berurutan dan merangkum hasil test. Tantangan utama adalah memastikan ID yang dihasilkan POST disimpan sebagai variabel environment dan digunakan oleh request berikutnya. Saya mengatasinya dengan menyimpan nilai data.id ke createdPolicyId dan menjalankan request sesuai urutan.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	Ya
2	Setiap endpoint API saya ada di Collection	Periksa setelah pengujian
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	Ya
4	Setiap request punya assertion status code dan struktur JSON	Periksa setelah pengujian
5	Nama setiap test berupa kalimat yang menyatakan harapan	Ya
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	Ya
7	Server dinyalakan ulang sebelum Runner dijalankan	Periksa setelah pengujian
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	Periksa setelah pengujian
9	Berkas Environment tidak memuat kunci rahasia sungguhan	Ya
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	Periksa setelah pengujian

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen